NEORV32 RISC-V Soft-Core CPU

FPGA Deployment Guide

Nexys4 (Artix-7 XC7A100T) Board

From Vivado Synthesis to Running C Programs

Including Permanent Flash Programming and Dual-OS Workflow

April 2026

Overview and Goals

This document provides a complete, step-by-step guide to deploying the NEORV32 RISC-V soft-core processor on a Digilent Nexys4 FPGA development board. It covers every phase from Vivado project setup through synthesizing the hardware, programming the FPGA, permanently storing the bitstream in SPI flash, and uploading and running C programs from Linux without ever needing Vivado again.

Every issue encountered during a real deployment is documented here, along with its root cause and exact solution, so that anyone following this guide can complete the process without facing unexpected failures.

Prerequisites

Requirement	Details
FPGA Board	Digilent Nexys4 (Legacy, no DDR) - Artix-7 XC7A100T
Vivado Version	Vivado 2018.2 or newer (Windows or Linux)
NEORV32 Repo	Cloned from github.com/stnolting/neorv32
RISC-V Toolchain	riscv32-unknown-elf-gcc or riscv64-unknown-elf-gcc
Serial Terminal	picocom (Linux) or PuTTY (Windows)
Python	Python 3 with pyserial (pip install pyserial)
USB Cable	Micro-USB for JTAG/UART (same cable for both)

System Architecture

NEORV32 is a RISC-V soft-core CPU written entirely in VHDL. It runs inside the FPGA fabric itself - there is no external CPU chip. The Nexys4 board provides a 100MHz clock, USB-to-UART bridge, LEDs, buttons, switches, and SPI flash memory. The deployment involves two separate concerns:

- Hardware deployment - synthesizing the CPU design and programming it into the FPGA
- Software deployment - compiling C programs with RISC-V GCC and uploading them via UART

Once the hardware is deployed and the bitstream is stored in SPI flash, hardware deployment never needs to be repeated. Only software deployment is needed for each new C program.

Section 1: Vivado Project Setup (Windows)

1.1 Important Decision: VHDL or Verilog

NEORV32 is written entirely in VHDL. You do not need to rewrite it or understand it. Vivado supports mixed-language projects, so you can write your top-level wrapper in Verilog if you prefer. However, the simplest approach is to use the VHDL test setup file provided by NEORV32 directly as your top level - it requires zero VHDL knowledge to use.

> **NOTE:** You do not write any RTL from scratch for this project. You use NEORV32 source files as-is and only provide pin constraints.

1.2 Creating the Vivado Project

1. Open Vivado and click Create Project
2. Choose RTL Project, uncheck Do not specify sources at this time
3. For part selection, use: xc7a100tcsg324-1 (Nexys4 legacy / no DDR version)
4. Do NOT select the board from the board catalog - select the part directly

> **WARNING:** The Nexys4 DDR and Nexys4 legacy boards use the same FPGA part number. Always verify xc7a100tcsg324-1.

1.3 Adding NEORV32 Source Files

Add all VHDL files from the NEORV32 rtl/core/ directory and the test setup file. In Vivado: Add Sources -> Add or Create Design Sources -> Add Files.

Required files:

- All .vhd files from neorv32/rtl/core/ (approximately 55 files)
- neorv32/rtl/test_setups/neorv32_test_setup_bootloader.vhd (the top level)

After adding files, right-click neorv32_test_setup_bootloader in the Sources panel and select Set as Top.

> **NOTE:** The default library must be set to neorv32. In Vivado: right-click the project in Sources -> Set Library -> neorv32

1.4 The Constraint File (XDC)

Create a new XDC file in Vivado: Add Sources -> Add or Create Constraints -> Create File. Name it neorv32_nexys4.xdc.

Paste the following complete constraint file. Every pin has been verified for the Nexys4 Legacy board:

```
## Clock - 100MHz
set_property PACKAGE_PIN E3 [get_ports clk_i]
set_property IOSTANDARD LVCMOS33 [get_ports clk_i]
create_clock -add -name sys_clk_pin -period 10.00 [get_ports clk_i]

## Reset - CPU Reset Button (active LOW on NEORV32)
set_property PACKAGE_PIN C12 [get_ports rstn_i]
set_property IOSTANDARD LVCMOS33 [get_ports rstn_i]
```

```
## UART TX (FPGA sends to PC via USB-UART bridge)
set_property PACKAGE_PIN D4 [get_ports uart0_txd_o]
set_property IOSTANDARD LVCMOS33 [get_ports uart0_txd_o]

## UART RX (PC sends to FPGA via USB-UART bridge)
set_property PACKAGE_PIN C4 [get_ports uart0_rxd_i]
set_property IOSTANDARD LVCMOS33 [get_ports uart0_rxd_i]

## LEDs (GPIO outputs)
set_property PACKAGE_PIN T8  [get_ports {gpio_o[0]}]
set_property PACKAGE_PIN V9  [get_ports {gpio_o[1]}]
set_property PACKAGE_PIN R8  [get_ports {gpio_o[2]}]
set_property PACKAGE_PIN T6  [get_ports {gpio_o[3]}]
set_property PACKAGE_PIN T5  [get_ports {gpio_o[4]}]
set_property PACKAGE_PIN T4  [get_ports {gpio_o[5]}]
set_property PACKAGE_PIN U7  [get_ports {gpio_o[6]}]
set_property PACKAGE_PIN U6  [get_ports {gpio_o[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {gpio_o[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {gpio_o[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {gpio_o[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {gpio_o[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {gpio_o[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {gpio_o[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {gpio_o[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {gpio_o[7]}]

## SPI Flash Boot Configuration (REQUIRED for permanent flash storage)
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property CONFIG_MODE SPIx4 [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCCO [current_design]
```

NOTE: The five BITSTREAM properties at the bottom are critical for SPI flash boot. Without them the FPGA cannot load its configuration from flash on power-up.

1.5 Running Synthesis and Implementation

5. Click Run Synthesis in the Flow Navigator and wait for completion
6. Click Run Implementation after synthesis succeeds
7. Click Generate Bitstream after implementation succeeds

The full flow takes 10-20 minutes depending on your machine. Timing should close easily for this design on xc7a100t.

WARNING: The warning 'The debug hub core was not detected' appears every time you program the board. This is completely harmless - NEORV32 has no ILA debug core and this warning can be permanently ignored.

Section 2: Programming the FPGA (Windows Vivado)

2.1 Programming the FPGA SRAM (Temporary)

This programs the FPGA's internal volatile SRAM. The configuration is lost when power is removed. Use this for quick testing before committing to flash.

8. Connect Nexys4 via USB and power on
9. In Vivado: Flow Navigator -> Hardware Manager -> Open Hardware Manager
10. Click Open Target -> Auto Connect
11. Right-click xc7a100t_0 -> Program Device
12. Select your .bit file and click Program
13. Wait for 'End of startup status: HIGH' in the TCL console

2.2 Setting the JP1 Jumper

The Nexys4 has a mode jumper JP1 located near the SD card slot that controls how the FPGA loads its configuration at power-up.

> **WARNING:** JP1 MUST be set to the QSPI position for the board to boot from SPI flash. If JP1 is in the wrong position, the flash will be programmed correctly but the board will not boot from it. This is the most common cause of flash programming appearing to succeed but the board not booting.

JP1 Position	Behavior
QSPI (correct)	FPGA loads bitstream from SPI flash on power-up
JTAG	FPGA waits for JTAG programming - will not auto-boot

2.3 Generating the MCS File for Flash Programming

The SPI flash requires an MCS format file, not the raw .bit file. Before generating it, verify the five BITSTREAM properties are in your XDC and regenerate the bitstream if you added them after the first synthesis.

In the Vivado TCL console, run:

```
write_cfgmem -format mcs \
  -interface SPIx4 \
  -size 128 \
  -loadbit {up 0x00000000 "E:/path/to/neorv32_test_setup_bootloader.bit"} \
  -file "E:/path/to/neorv32_nexys4.mcs" \
  -force
```

OR in GUI you can follow these steps

Tools → Generate Memory Configuration File

→ Format: MCS

→ Interface: SPIx4

→ select your .bit file

→ generate .mcs file

Then:

Hardware Manager → Add Configuration Memory Device

→ select s25fl128sxxxxxx0-spi-x1-x2-x4

→ Program Configuration Memory Device

→ select your .mcs file

Expected output: INFO: [Writecfgmem 68-12] Done.

If all done then now your hardware is stored in flash no need to again and again upload bitstream after the power cut.

> **WARNING:** A common error at this step is: SPI_BUSWIDTH property is set to 1. This means the five BITSTREAM properties were not in the XDC when the bitstream was generated. Add them to the XDC, regenerate the bitstream, then retry this command.

2.4 Known Issue: Vivado Flash Programmer Failure

Vivado 2018.2 has a known bug where the program_hw_cfgmem command consistently fails with 'Failure to set flash parameters' on this board. This is a Vivado bug, not a user error. The solution is to use OpenOCD on Linux instead of Vivado for flash programming.

Copy the generated files to Linux before proceeding:

- neorv32_test_setup_bootloader.bit
- neorv32_nexys4.mcs

Note: If you did not find any issue and the FPGA is programmed successfully then skip section 3 and go to section 4 otherwise follow the section 3 also.

Section 3: Permanent Flash Programming (Linux)

This section permanently stores the NEORV32 bitstream in the Nexys4's onboard SPI flash chip (Spansion s25fl128s). After this, the FPGA configures itself automatically on every power-on without any laptop or software needed.

3.1 Why This Is Necessary

The FPGA's internal SRAM is volatile - it loses its configuration when power is removed. The Nexys4 has a 128Mb SPI flash chip specifically for storing the bitstream permanently. Once programmed, the FPGA reads the bitstream from flash on every power-up, making the board completely standalone.

This is especially important for dual-boot or multi-laptop setups where the board needs to be ready without re-programming.

3.2 Setting the JP1 Jumper

The Nexys4 has a mode jumper JP1 located near the SD card slot that controls how the FPGA loads its configuration at power-up.

> **WARNING:** JP1 MUST be set to the QSPI position for the board to boot from SPI flash. If JP1 is in the wrong position, the flash will be programmed correctly but the board will not boot from it. This is the most common cause of flash programming appearing to succeed but the board not booting.

JP1 Position	Behavior
QSPI (correct)	FPGA loads bitstream from SPI flash on power-up
JTAG	FPGA waits for JTAG programming - will not auto-boot

3.3 Installing Required Tools

```
# Install OpenOCD
sudo apt install openocd

# Verify installation
openocd --version

# Download the BSCAN SPI proxy bitfile for xc7a100t
wget
https://github.com/quartiq/bscan_spi_bitstreams/raw/master/bscan_spi_xc7a100t.bit \
     -O ~/bscan_spi_xc7a100t.bit
```

The bscan_spi_xc7a100t.bit file is a small proxy bitfile that OpenOCD loads into the FPGA to gain access to the SPI flash through the JTAG interface. It is not your NEORV32 bitstream - it is a helper tool.

3.4 Converting BIT to BIN Format

OpenOCD requires the raw binary bitstream, not the Vivado .bit format. The .bit file has a header that must be stripped. Run this Python script to strip the header:

```
python3 - <<'EOF'
with open('neorv32_test_setup_bootloader.bit', 'rb') as f:
    data = f.read()

# Xilinx sync word marks the start of actual bitstream data
sync = bytes([0xAA, 0x99, 0x55, 0x66])
idx = data.find(sync)
print(f'Sync word found at offset: {idx}')

with open('neorv32_test_setup_bootloader.bin', 'wb') as f:
    f.write(data[idx:])

print(f'Written {len(data)-idx} bytes to .bin file')
EOF
```

3.5 Creating the OpenOCD Configuration File

Create the file ~/nexys4_flash.cfg with the following content:

```
adapter driver ftdi
ftdi vid_pid 0x0403 0x6010
ftdi channel 0
ftdi layout_init 0x00e8 0x60eb
adapter speed 10000

transport select jtag

source [find cpld/xilinx-xc7.cfg]
source [find cpld/jtagspi.cfg]

init
puts "JTAG initialized"

jtagspi_init 0 {/home/YOUR_USERNAME/bscan_spi_xc7a100t.bit}
puts "Flash initialized"

jtagspi_program {/home/YOUR_USERNAME/ITU/neorv32_test_setup_bootloader.bin} 0x0
puts "Flash programmed!"

shutdown
```

WARNING: Replace YOUR_USERNAME with your actual Linux username in both file paths. The paths must be absolute.

3.6 Running Flash Programming

```
sudo openocd -f ~/nexys4_flash.cfg
```

Expected output shows sector-by-sector programming (approximately 59 sectors) and ends with:

```
Info : Found flash device 'sp s25fl128s' (ID 0x182001)
Flash initialized
Info : sector 0 took 1928 ms
Info : sector 1 took 1909 ms
```

```
...
Info : sector 58 took 114 ms
Flash programmed!
shutdown command invoked
```

The process takes approximately 3-5 minutes. Do not disconnect the board during programming.

3.7 Verifying Successful Flash Programming

14. Power off the board using the POWER slide switch
15. Wait 5 seconds
16. Power on the board
17. Open picocom immediately: sudo picocom -b 19200 --flow n /dev/ttyUSB1
18. The NEORV32 bootloader message should appear automatically within 2 seconds

If the bootloader appears without any JTAG programming, flash programming was successful. The board is now permanently configured.

Section 4: UART Communication and Bootloader

4.1 Understanding the NEORV32 Bootloader

NEORV32 boots into a built-in bootloader stored in a dedicated boot ROM inside the FPGA. The bootloader runs at 19200 baud and provides a menu for uploading programs. It gives 8 seconds for user input before attempting to auto-boot any stored program.

Bootloader Command	Action
h	Show help menu with all available commands
u	Upload executable via UART (send neorv32_exe.bin)
e	Execute the previously uploaded program
i	Show system information (clock, ISA extensions)
r	Restart the bootloader
l	Load program from SPI flash
s	Store program to SPI flash

4.2 Serial Terminal Setup

Use picocom on Linux. Do not use VS Code serial monitor - it has issues sending keystrokes to hardware bootloaders.

```
sudo apt install picocom
sudo picocom -b 19200 --flow n /dev/ttyUSB1
```

Key picocom settings:

- Baud rate: 19200 (NEORV32 bootloader default)
- Flow control: none (--flow n is critical - flow control blocks TX)
- Data bits: 8, Stop bits: 1, Parity: none
- To exit picocom: Ctrl+A then Ctrl+X

NOTE: Only one program can use the serial port at a time. Always close picocom before running the upload script, and vice versa.

4.3 Known Issue: Bootloader Not Responding to Keypress

Symptom: You can see the bootloader output (RX works) but pressing keys has no effect (TX does not work).

Root cause: Flow control is enabled by default on Linux serial ports. This blocks transmit while receive works fine - exactly matching the symptom.

Solution: Always use --flow n with picocom to explicitly disable flow control.

Secondary cause: Another program (VS Code, screen, minicom) has the serial port open and is stealing the data. Check with:

```
sudo lsof /dev/ttyUSB1
```

Kill any listed process before opening picocom.

4.4 Finding the Correct Serial Port

```
ls /dev/ttyUSB*
```

The Nexys4 USB-UART bridge typically appears as /dev/ttyUSB1. If you see multiple ports, try each one. The correct port shows bootloader output when the board is reset.

Section 5: Uploading and Running C Programs

5.1 The Upload Python Script

The NEORV32 provided bash upload script (uart_upload.sh) has timing issues that cause it to fail. Use this Python script instead, which handles timing correctly:

Create the file ~/upload.py with the following content:

```
import serial
import time
import sys

port = sys.argv[1]
binfile = sys.argv[2]

ser = serial.Serial(port, 19200, timeout=2)
time.sleep(0.5)

print('Waiting for bootloader...')
ser.reset_input_buffer()

# Send space to abort autoboot
ser.write(b' ')
time.sleep(0.5)

# Wait for CMD:> prompt
response = ser.read(100).decode(errors='ignore')
print(f'Got: {repr(response)}')

# Send u for upload
print('Sending upload command...')
ser.write(b'u')
time.sleep(0.5)

response = ser.read(50).decode(errors='ignore')
print(f'Got: {repr(response)}')

# Send the binary file
print('Sending binary...')
with open(binfile, 'rb') as f:
    data = f.read()

ser.write(data)
print(f'Sent {len(data)} bytes')

# Wait for OK response
time.sleep(3)
response = ser.read(200).decode(errors='ignore')
print(f'Response: {repr(response)}')

if 'OK' in response:
    print('Upload successful! Executing...')
    ser.write(b'e')
    time.sleep(2)
```

```
        output = ser.read(500).decode(errors='ignore')
        print('Program output:')
        print(output)
    else:
        print('Upload failed or timed out')
        print('Full response:', repr(response))

    ser.close()
```

Install the required Python package:

```
pip install pyserial
# or
sudo apt install python3-serial
```

5.2 Reset Button Behavior

The NEORV32 reset signal (rstn_i) is active LOW. This means holding the reset button keeps the CPU in reset, and releasing it starts execution. This is the opposite of what many MCU users expect.

The correct upload procedure:

19. Hold the reset button on the board
20. Run the upload script
21. When the script prints 'Waiting for bootloader...' and shows the Got: response, release the reset button

The script sends a space character immediately to abort autoboot, then sends 'u' to start upload. Releasing reset at the right moment allows the bootloader to start and catch the space character.

> **NOTE:** If you release the reset button before running the script, the 8-second autoboot window may expire. Hold reset, start the script, then release. Or you can use the button which behaves as ACTIVE LOW or just change the top module to active high instead of active low

5.3 Running the Upload

```
# Install pyserial if not already installed
pip install pyserial

# Hold reset button, then run:
python3 ~/upload.py /dev/ttyUSB1 /path/to/neorv32_exe.bin

# Release reset button after script starts
```

Successful output looks like:

```
Waiting for bootloader...
Got: "Aborted.\r\n\r\nType 'h' for help.\r\nCMD:> "
Sending upload command...
Got: 'u\r\nAwaiting neorv32_exe.bin... '
Sending binary...
Sent 5508 bytes
Response: 'OK\r\nCMD:> '
Upload successful! Executing...
Program output:
```

Hello world! :)

Section 6: Writing and Building C Programs

6.1 Project Structure Outside NEORV32 Directory

You can create your own C programs in any directory without modifying the NEORV32 repository. The NEORV32 common makefile handles all compilation details automatically.

```
mkdir -p ~/myprojects/my_program
cd ~/myprojects/my_program
```

6.2 The Makefile

Create a file named makefile (lowercase) in your project directory:

```
# Path to NEORV32 root - adjust to your installation
NEORV32_HOME ?= $(HOME)/ITU/neorv32

# Your source files
APP_SRC = main.c

# Include NEORV32 common makefile
include $(NEORV32_HOME)/sw/common/common.mk
```

6.3 Example: LED Blink with UART Output

```
#include <neorv32.h>

int main(void) {

    // Initialize UART at 19200 baud (must match bootloader)
    neorv32_uart0_setup(19200, 0);
    neorv32_uart0_printf("My NEORV32 program starting!\n");

    // Initialize GPIO - all LEDs off
    neorv32_gpio_port_set(0);

    uint32_t led = 1;

    while (1) {
        neorv32_gpio_port_set(led);       // turn on one LED

        // neorv32_aux_delay_ms requires clock frequency as first argument
        neorv32_aux_delay_ms(100000000, 300);  // 300ms delay at 100MHz

        led = led << 1;                   // shift to next LED
        if (led > 0xFF) led = 1;          // wrap after LED7
    }

    return 0;
}
```

6.4 Building

```
make clean all
```

Successful build output ends with:

```
Memory utilization:
   text    data     bss     dec     hex filename
   5492       0     256    5748    1674 main.elf
Generating neorv32_exe.bin
Executable (EXE): 5492 bytes @ 0x00000000
```

The file neorv32_exe.bin in your project directory is the file to upload to the board.

6.5 Common Build Error: Wrong Function Signature

Error: too few arguments to function neorv32_aux_delay_ms

Cause: This function requires the clock frequency as its first argument.

Wrong:

```
neorv32_aux_delay_ms(300);
```

Correct:

```
neorv32_aux_delay_ms(100000000, 300);  // 100MHz clock, 300ms delay
```

6.6 Program Memory Limits

Memory	Default Size
IMEM (program memory)	16 KB
DMEM (data/stack/heap)	8 KB
Boot ROM (bootloader)	Fixed, read-only

16KB is sufficient for most embedded C programs. To increase IMEM, modify the MEM_INT_IMEM_SIZE generic in neorv32_test_setup_bootloader.vhd and regenerate the bitstream.

Section 7: Daily Development Workflow

7.1 Single Linux Machine

Once flash programming is complete:

22. Plug in board - NEORV32 boots automatically from flash
23. Write your C program
24. Compile: make clean all
25. Hold reset button on board
26. Run: python3 ~/upload.py /dev/ttyUSB1 neorv32_exe.bin
27. Release reset button
28. See output in script or open picocom

7.2 Dual-Boot Windows/Linux Setup

Since the bitstream is permanently stored in flash, switching between Windows and Linux via Restart (not Shutdown) keeps the board programmed:

29. Boot into Linux
30. Plug in board - NEORV32 auto-boots from flash
31. Compile and upload C programs normally
32. When done, Restart into Windows (not Shutdown)

NOTE: Use Restart, not Shutdown. A restart does not cut power to the USB ports on most machines, so the FPGA retains its SRAM configuration. However, even if power cuts during shutdown, the flash ensures the board auto-configures on next power-up.

7.3 Multi-Laptop Setup

The board works with any Linux laptop. Copy to the second laptop:

- ~/upload.py (the upload script)
- Your project directories under ~/myprojects/
- The NEORV32 repo (or just the sw/ folder) for the library headers

The second laptop needs only:

```
# RISC-V toolchain
sudo apt install gcc-riscv64-unknown-elf

# Python serial library
pip install pyserial

# picocom for serial monitor
sudo apt install picocom
```

The board itself needs nothing from the laptop - it boots NEORV32 on its own.

Section 8: Complete Troubleshooting Reference

8.1 Vivado Issues

Debug hub warning on every programming

Message: WARNING: [Labtools 27-3361] The debug hub core was not detected

Cause: NEORV32 has no Vivado ILA debug core. This warning is harmless.

Solution: Ignore it completely. It does not affect functionality.

SPI_BUSWIDTH error when generating MCS

Message: SPI_BUSWIDTH property is set to 1. This property has to be set to 4

Cause: The five BITSTREAM properties were missing from the XDC when synthesis ran.

Solution: Add the five properties to the XDC, re-run Generate Bitstream, then retry write_cfgmem.

program_hw_cfgmem fails with Failure to set flash parameters

Cause: Known Vivado 2018.2 bug with this specific flash chip. Not a user error.

Solution: Use OpenOCD on Linux as described in Section 3. Do not waste time debugging Vivado flash programming.

Add Configuration Memory Device is grayed out

Cause: The FPGA was not programmed in the current session, or the Vivado session is in a bad state.

Solution: Program the FPGA first with program_hw_devices, then immediately right-click xc7a100t_0. If still grayed, use the TCL console approach or switch to OpenOCD on Linux.

8.2 Serial Communication Issues

Bootloader output visible but keypress has no effect

Cause: Flow control is enabled on the serial port, blocking TX while RX works.

Solution: Always use picocom with --flow n flag. Never use VS Code serial monitor for bootloader interaction.

Upload script returns empty response

Cause: Another program has the serial port open (picocom, VS Code, screen, minicom).

Solution: Run sudo lsof /dev/ttyUSB1 and kill any listed process before running the upload script.

uart_upload.sh times out

Cause: The bash script has timing issues - it sends bytes too fast without waiting for bootloader acknowledgment.

Solution: Use the Python upload script from Section 5.1 instead of uart_upload.sh.

No executable. Boot anyway? (y/n) after reset

Cause: The bootloader lost its valid-program flag after reset. The program itself is still in IMEM.

Solution: Press y - the board jumps to address 0x00000000 where your program still sits and runs it normally. This is expected behavior, not an error.

8.3 Flash Programming Issues

Board does not auto-boot after flash programming

Most likely cause: JP1 jumper is not in QSPI position.

Solution: Locate JP1 near the SD card slot on the Nexys4. Set it to QSPI position. Power cycle the board.

OpenOCD shows unknown JEDEC manufacturer: ff

Cause: The bscan_spi proxy bitfile failed to load into the FPGA, so the flash chip is not accessible.

Solution: This was caused by using the wrong bscan_spi bitfile variant (xa7a100t vs xc7a100t). Use the correct file: bscan_spi_xc7a100t.bit from the quartiq/bscan_spi_bitstreams repository.

xc3sprog silently fails or shows no output

Cause: xc3sprog version from Ubuntu package is from 2011 and has compatibility issues with newer flash chips.

Solution: Use OpenOCD instead of xc3sprog for all flash programming operations.

Flash verification shows contents differ

Cause: The MCS file was passed to OpenOCD which treated it as raw binary. MCS is Intel HEX format and must be converted.

Solution: Convert .bit to .bin using the Python conversion script in Section 3.4. Use the .bin file for OpenOCD, not the .mcs file.

Section 9: Quick Reference

File Reference

File	Purpose / Notes
neorv32_test_setup_bootloader.vhd	NEORV32 top level - set as top in Vivado
neorv32_nexys4.xdc	Pin constraints - must include 5 BITSTREAM properties
neorv32_test_setup_bootloader.bit	Vivado bitstream output
neorv32_nexys4.mcs	Flash image (Intel HEX format) - generated by write_cfgmem
neorv32_test_setup_bootloader.bin	Raw binary for OpenOCD - extracted from .bit file
bscan_spi_xc7a100t.bit	OpenOCD proxy bitfile - downloaded from GitHub
nexys4_flash.cfg	OpenOCD configuration - created once, reused forever
upload.py	Python upload script - used for every program upload
neorv32_exe.bin	Compiled program output - generated by make clean all

Key Parameters

Parameter	Value
FPGA Part	xc7a100tcsg324-1
Clock Frequency	100 MHz (100000000 Hz)
Bootloader Baud Rate	19200
Serial Port (Linux)	/dev/ttyUSB1 (verify with ls /dev/ttyUSB*)
USB VID:PID	0x0403:0x6010 (FTDI chip on Nexys4)
SPI Flash Chip	Spansion s25fl128s (128Mb)
Flash JEDEC ID	0x182001
JP1 Setting for Flash Boot	QSPI
IMEM Default Size	16 KB
DMEM Default Size	8 KB

Complete Command Reference

Flash Programming (run once)

```
# Download bscan proxy
wget
https://github.com/quartiq/bscan_spi_bitstreams/raw/master/bscan_spi_xc7a100t.bit -
O ~/bscan_spi_xc7a100t.bit

# Convert .bit to .bin
python3 bit_to_bin.py  # see Section 3.4

# Program flash
sudo openocd -f ~/nexys4_flash.cfg
```

Daily Development

```
# Compile
cd ~/myprojects/my_program && make clean all

# Upload (hold reset button first)
python3 ~/upload.py /dev/ttyUSB1 neorv32_exe.bin

# Monitor serial output
sudo picocom -b 19200 --flow n /dev/ttyUSB1

# Exit picocom
Ctrl+A then Ctrl+X
```

Diagnostics

```
# Find serial port
ls /dev/ttyUSB*

# Check who is using the port
sudo lsof /dev/ttyUSB1

# Detect JTAG chain
sudo xc3sprog -c nexys4 -j

# Check serial port settings
stty -F /dev/ttyUSB1
```

End of NEORV32 FPGA Deployment Guide